

# TruffleHog Secret Scanning

## Integration with Jenkins CI/CD Pipeline

*Setup Guide, Configuration Reference & Proof of Concept*

### Document purpose

This document records the complete setup of TruffleHog (secret-scanning tool) on a Jenkins build agent, its integration into a Jenkins declarative pipeline, and a proof of concept demonstrating that the pipeline correctly blocks a build when a hardcoded AWS credential is found in the source repository.

## 1. Overview

---

TruffleHog is an open-source secret-scanning tool that searches source code repositories, commit history, and file systems for accidentally committed credentials such as AWS keys, API tokens, and private keys. In this setup, TruffleHog is installed on the Jenkins build agent and wired into the pipeline as a dedicated "Secret Scan" stage that runs before the Build and Deploy stages.

Pipeline flow used in this setup:

1. Initialize – basic environment checks (PATH, M2\_HOME).
2. Secret Scan (TruffleHog) – scans the GitHub repository for hardcoded secrets.
3. Build – runs ‘mvn clean package’ (only if no secrets were found).
4. Deploy to Tomcat – copies the built WAR file to the Tomcat server over SSH (only if Build succeeded).

The key design decision: if TruffleHog reports a detected secret, the pipeline stage throws an error and the pipeline stops immediately — the Build and Deploy stages are skipped entirely. This prevents any commit containing a hardcoded credential from ever reaching the Tomcat server.

## 2. Prerequisites

---

- An Ubuntu-based Jenkins build agent (EC2 instance in this setup) with sudo access.
- Jenkins installed and running, with a service account named ‘jenkins’.
- Internet access on the agent to download the TruffleHog release binary.
- A GitHub repository containing the application source code.
- SSH credentials configured in Jenkins (credential ID ‘tomcat’) for deployment to the Tomcat host.

## 3. Installing TruffleHog on the Jenkins Agent

---

Install the TruffleHog binary system-wide so both the ‘ubuntu’ login user and the ‘jenkins’ service user can access it.

### 3.1 Download and install to /usr/local/bin

```
curl -sSfL https://raw.githubusercontent.com/trufflesecurity/\
trufflehog/main/scripts/install.sh | sh -s -- -b /usr/local/bin
```

This installs the binary to /usr/local/bin/trufflehog, which is on the default PATH for all system users, including the Jenkins service account.

### 3.2 Verify the installation

```
which trufflehog
trufflehog --version

# Confirm the Jenkins service account can also see it
sudo -u jenkins trufflehog --version
```

### 3.3 Known issue: old cached path in an existing shell session

If TruffleHog was previously installed in a user-local path (for example ~/.local/bin/trufflehog) and later reinstalled to /usr/local/bin, an already-open terminal session may still resolve the old, now-missing path. Bash caches resolved command paths for the lifetime of the session (‘hashing’).

#### Symptom

```
-bash: /home/ubuntu/.local/bin/trufflehog: No such file or directory
```

Fix – clear bash’s cached path, or simply open a new terminal session:

```
hash -r
trufflehog --version

# or, in a fresh terminal:
which trufflehog
trufflehog --version
```

### 3.4 Known issue: auto-updater error on scan

TruffleHog checks for a newer release and tries to self-update on each run. If the executing user does not have write permission on the binary’s location, this fails with a warning — the scan itself is unaffected.

#### Symptom

```
{"level":"error", ... "msg":"error occurred with trufflehog updater", "error":"cannot move binary (exit status 1)"}
```

Two ways to resolve this:

- Recommended – disable the auto-updater explicitly for every scan by adding the ‘--no-update’ flag.
- Alternative – grant the executing user ownership of the binary so it can update itself: ‘sudo chown ubuntu:ubuntu /usr/local/bin/trufflehog’. (Not used in the final Jenkinsfile, since Jenkins runs scans non-interactively and ‘--no-update’ is the cleaner, repeatable fix.)

```
trufflehog github --repo=https://github.com/<org>/<repo>.git --json --no-update
```

## 4. Running a Manual Scan (Command Reference)

Core command used to scan a GitHub repository and print results as JSON:

```
trufflehog github --repo=https://github.com/Shubham-sys-web/webapp.git --json --no-update
```

Sample output on a clean commit (no secrets present):

```
{"level":"info-0","msg":"running source", ...}
{"level":"info-0","msg":"Completed enumeration","num_repos":1}
{"level":"info-0","msg":"scanning repo", ...}
{"level":"info-0","msg":"finished scanning","chunks":357,"bytes":51836,
  "verified_secrets":0,"unverified_secrets":0,"scan_duration":"1.96s"}
```

Sample output once a hardcoded credential is present in the repository:

```
{"SourceMetadata":{"Data":{"Git":{"commit":"...", "file":"index.jsp",
  "line":13, "repository":"https://github.com/Shubham-sys-web/webapp.git"}}},
  "SourceID":1, "SourceType":7, "SourceName":"trufflehog - github",
  "DetectorType":2, "DetectorName":"AWS",
  "Verified":false,
  "Raw":"AKIAZQ3XJ9K2LMNPQRST",
  "ExtraData":{"resource_type":"Access key"}}
```

The presence of the ‘DetectorName’ field is what indicates a real detection — this is the field the Jenkins pipeline logic checks for.

## 5. Important Note: Choosing a Test Secret

### Why the AWS documentation example key does not get flagged

AKIAIOSFODNN7EXAMPLE is AWS's own official placeholder key, used across their documentation, SDKs, and countless tutorials. Because it appears so widely as a known, non-sensitive placeholder, TruffleHog (and other scanners such as Gitleaks and git-secrets) explicitly ignore it as a documented false positive to avoid noisy alerts. This is expected, correct scanner behaviour — not a bug.

For testing purposes, use a fake key that matches AWS's real key format (AKIA + 16 alphanumeric characters) but is not a published placeholder, for example:

```
<!--  
  TEST SECRET FOR TRUFFLEHOG SCANNING - DO NOT USE IN PRODUCTION  
  AWS_ACCESS_KEY_ID=AKIAZQ3XJ9K2LMNPQRST  
  AWS_SECRET_ACCESS_KEY=k3F9mP2xQ8vN5tR7wY1zA4bC6dE0gH3jK5lM8nO2  
-->
```

This format-matches a real AWS key closely enough for the detector's pattern and entropy checks to trigger, while being a unique string that is not on any scanner's ignore list. Because the key is fake, it will show up as an unverified secret (TruffleHog cannot verify it against the live AWS API), which is the expected and desired result for a test.

## 6. Jenkins Pipeline Configuration

The full declarative Jenkinsfile below integrates the secret scan as a gating stage ahead of Build and Deploy.

```
pipeline {  
  agent any  
  stages {  
  
    stage('Initialize') {  
      steps {  
        sh '''  
          echo "PATH = ${PATH}"  
          echo "M2_HOME = ${M2_HOME}"  
        '''  
      }  
    }  
  
    stage('Secret Scan - TruffleHog') {  
      steps {  
        script {  
          sh '''  
            trufflehog github \  
              --repo=https://github.com/Shubham-sys-web/webapp.git \  
              --json --no-update \  
            > trufflehog-report.json || true  
          '''  
          sh 'cat trufflehog-report.json'  
  
          def report = readFile('trufflehog-report.json')  
          if (report.contains('"DetectorName"')) {  
            echo "SECRETS DETECTED by TruffleHog! Build aborted."  
            error "Pipeline stopped: hardcoded secrets found."  
          } else {  
            echo "No secrets detected. Proceeding to build."  
          }  
        }  
      }  
    }  
  
    stage('Build') {  
      steps {  
        sh 'mvn clean package'  
      }  
    }  
  }  
}
```

```

stage('Deploy To Tomcat') {
    steps {
        sshagent(['tomcat']) {
            sh '''
                scp -o StrictHostKeyChecking=no \
                    target/*.war \
                    ubuntu@13.233.1.94:/opt/tomcat/webapps/
            '''
        }
    }
}

```

### 6.1 Why ‘report.contains("DetectorName")’ and not just ‘file not empty’

An earlier version of this check tested only whether trufflehog-report.json had content (`readFile(...).trim().length() > 0`). That is unreliable, because TruffleHog can write informational log lines (level ‘info-0’ status messages) even when zero secrets are found, so the file is never actually empty. Checking specifically for the ‘DetectorName’ field is accurate, because that field is only present in the JSON when an actual secret match has been reported.

### 6.2 Deployment steps in Jenkins

1. Open the Jenkinsfile in the GitHub repository and replace its contents with the pipeline above.
2. Commit the change to the repository.
3. In Jenkins, open the pipeline job and click ‘Build Now’.
4. Open ‘Console Output’ or the Blue Ocean pipeline view to observe stage-by-stage execution.

## 7. Proof of Concept: Pipeline Blocks a Build on Detected Secret

The test commit added a fake AWS access key (AKIAZQ3XJ9K2LMNPQRST, format-matched but not a real credential) to index.jsp, then triggered a Jenkins build. The following two screenshots, taken from the resulting build, confirm the pipeline behaved as designed.

### 7.1 Console Output – Detection and Abort

The console log shows TruffleHog's JSON output for the scan, including the detected AWS key details (SourceType, DetectorName: "AWS", the raw key value, and structured access-key data). Immediately after, the pipeline prints the “SECRETS DETECTED” message, and the Build and Deploy To Tomcat stages are both explicitly marked as “skipped due to earlier failure(s).” The pipeline finishes with status FAILURE.

```

42857798}}}, {"SourceID":1, "SourceType":7, "SourceName":"trufflehog - github", "DetectorType":2, "DetectorName":"AWS", "DetectorDescription":"AWS (Amazon Web Services) is a comprehensive cloud computing platform offering a wide range of on-demand services like computing power, storage, databases. API keys for AWS can have varying amount of access to these services depending on the IAM policy", "Attached":true, "DecoderName":"PLAIN", "Verified":false, "VerificationFromCache":false, "Raw":"AKIAZQ3XJ9K2LMNPQRST", "RawV2":"AKIAZQ3XJ9K2LMNPQRST:k3F9mP2xQ8vN5tr7wY1zA4bC6dE0gH3jK5lM8nO2", "Redacted":"AKIAZQ3XJ9K2LMNPQRST", "ExtraData":{"resource_type":"Access key"}, "StructuredData":null, "SecretParts":[{"access_key_id":"AKIAZQ3XJ9K2LMNPQRST", "secret_access_key":"k3F9mP2xQ8vN5tr7wY1zA4bC6dE0gH3jK5lM8nO2"}]}
[Pipeline] endOfPipeline
[Pipeline] echo
[Pipeline] error
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
Stage "Build" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy To Tomcat)
Stage "Deploy To Tomcat" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: Pipeline stopped: hardcoded secrets found in repository.
Finished: FAILURE

```

Figure 1 – Jenkins console output: TruffleHog detection (AWS key) followed by pipeline abort and skipped downstream stages.

## 7.2 Blue Ocean View – Pipeline Stage Graph

The Blue Ocean visual pipeline view shows the same result graphically: ‘Initialize’ completed successfully (green), ‘Secret Scan - TruffleHog’ failed (red), and both ‘Build’ and ‘Deploy To Tomcat’ remained un-run (grey/open circles) because the pipeline halted before reaching them. The step log underneath confirms the ‘SECRETS DETECTED by TruffleHog! Build aborted.’ message and the final error signal ‘Pipeline stopped: hardcoded secrets found in repository.’

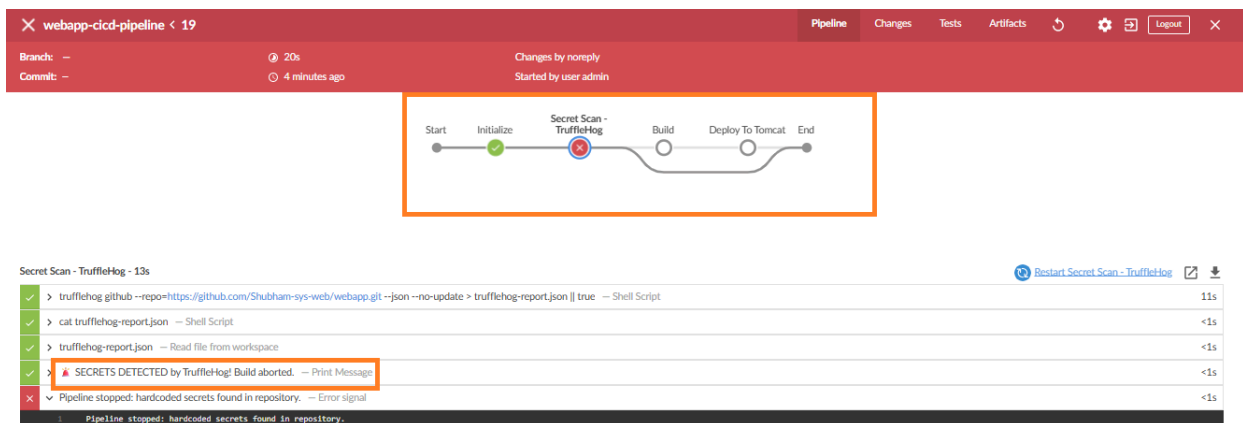


Figure 2 – Blue Ocean pipeline graph: Secret Scan stage failed; Build and Deploy To Tomcat never executed.

## 7.3 Result Summary

**POC outcome: PASS**

A hardcoded AWS-format credential committed to the repository was detected by the TruffleHog scan stage, the pipeline stopped with a controlled error before compiling or deploying any code, and no artifact built from the vulnerable commit was deployed to the Tomcat server.

## 8. Operational Notes

---

- This check currently flags any detection regardless of verification status ('Verified': false is still treated as a blocking finding). This is intentional — unverified does not mean harmless, it only means TruffleHog could not confirm the credential against the live provider API.
- Remove or rotate any real secret immediately if TruffleHog ever reports a 'Verified': true detection — that indicates a live, working credential was found in the repository.
- Keep the '--no-update' flag in all Jenkins-triggered scans so pipeline runs are not affected by TruffleHog's self-update mechanism or transient network issues.
- Consider archiving trufflehog-report.json as a Jenkins build artifact for audit history.
- This setup scans the full repository on every run via the GitHub source connector; for large repositories, scope or caching options can be evaluated to reduce scan time.

---

*End of document.*